

WEEK 02

Web Development

AI in the repository

Software Lab II · 103411 · 2026/27

Pedro Riera Martorell · Jordi López Galera · Tecnocampus UPF



Centre universitari adscrit a la



Today's session

0-10' Why configure AI in the repository

10-25' The main file: AGENTS.md

25-40' Skills

40-50' Agents, prompts, tools and hooks

50-60' Providers: Copilot, Claude Code, Windsurf...

60-85' Hands-on: configure your team repo

85-90' Review and questions

SECTION 01

The main file

What goes in, what stays out

Put in

- Build, test and run commands
- Short architecture map
- Naming and code conventions
- Branch and PR workflow
- Mistakes the agent repeats

Leave out

- Secrets, tokens, passwords
- Long docs (link them instead)
- What the linter already enforces
- Generic advice: “write clean code”
- Outdated information

Every line is read in every session: short and specific beats long and vague.

A new teammate

The AI agent joins your team
every day with no memory
of the project.

- Doesn't know your stack, commands or conventions
- Reads the repository from scratch each session
- Missing context → it guesses
- Repository configuration = onboarding docs for the agent

Layers of context

Layer	When it is loaded	Example
Instructions	Always	“We use squash merge and Conventional Commits”
Scoped rules	When matching files are touched	“How we write React tests”
Skills	When the task needs it	“How to add a REST endpoint”
Agents	When work is delegated	“Read-only PR reviewer”
Tools (MCP)	When the agent needs to act outside	GitHub issues, database, browser
Hooks	Always, automatically (not AI)	Run the linter after every edit

AGENTS.md

```
protube/
├── AGENTS.md           ← main file
├── backend/
│   ├── AGENTS.md      ← Spring Boot rules
│   └── src/...
├── frontend/
│   ├── AGENTS.md      ← React rules
│   └── src/...
└── .github/workflows/
```

- Plain Markdown, no special syntax
- Lives in the repo, versioned with the code
- Loaded in every session (always on)
- Nested files for sub-projects: the closest one wins
- Open standard read by most agents

Example

Protube

Video platform. Monorepo:

backend/ (Spring Boot, Java 21)

frontend/ (React + TypeScript, Vite)

Commands

- Backend: `cd backend && ./mvnw verify`
- Frontend: `cd frontend && npm test`

Conventions

- Branch: `feature/<ticket>-short-name`
- Commits: Conventional Commits
- Never push to main: PR + squash
- Controller → Service → Repository
- Return DTOs, never JPA entities

Definition of Done

- Tests added, CI green
- Coverage threshold met

- What the project is (2 lines)
- Exact commands to build and test
- Conventions it can't guess
- Workflow rules: branches, PRs
- Definition of Done

Scrum: the DoD lives next to the code

Scoped rules

```
.github/instructions/  
  react-tests.instructions.md  
---  
applyTo: "frontend/**/*.test.tsx"  
---  
- Query by role (Testing Library)  
- Mock the API, never call backend  
- One describe per component  
- Run: npm test -- --coverage
```

- Rules for some paths only (glob pattern)
- Loaded only when those files are touched
- Saves context for everything else
- Same idea everywhere: nested AGENTS.md / CLAUDE.md, Cursor .mdc globs
- Windsurf: rules with a “glob” trigger

SECTION 02

Skills

What is a skill?

1

Name + description

Always visible: one line per skill

2

SKILL.md body

Loaded when the task matches

3

Scripts, templates, examples

Loaded only if a step needs them

- A folder that teaches one repeatable task
- The agent decides when to use it, from the description
- Can also be called by name
- Open format: the same SKILL.md works in many tools

Anatomy of a skill

Folder (also .claude/skills/ or .agents/skills/)

```
.github/skills/  
└─ create-rest-endpoint/  
    │ SKILL.md  
    │ templates/  
    │ │ Controller.java  
    │ │ └─ ControllerTest.java  
    └─ scripts/  
        └─ check-openapi.sh
```

SKILL.md

name: create-rest-endpoint
description: Create a REST endpoint
in the Spring Boot backend
(controller, service, DTO, tests).
Use when adding an API route.

1. Follow layering in backend/AGENTS.md
2. Start from templates/
3. Add a MockMvc test
4. Run ./mvnw test until green
5. Run scripts/check-openapi.sh

Creating a skill

1

Spot it

You explained the same task twice? That is a skill.

2

Describe it

Name + description: what it does and when to use it.

3

Write steps

Short, imperative, numbered. Point to files.

4

Add resources

Templates, scripts, good examples.

5

Test in a PR

Try it, fix it, a teammate reviews it.

Tip: most agents can draft a first version: “Create a skill for how we add a REST endpoint”. Then you review it.

SECTION 03

Agents

Instructions, skill or agent?

	Instructions	Skill	Agent
Metaphor	House rules	Playbook	Specialist teammate
Loaded	Always	When relevant	When delegated
Context	Shared	Shared (main chat)	Its own, isolated
Tools	All allowed	All + its scripts	A restricted list
Example	“Squash merge only”	“Create REST endpoint”	“Read-only reviewer”

A custom agent

```
.github/agents/reviewer.agent.md
---
name: reviewer
description: Reviews the branch diff
            against our conventions. Read-only.
tools: ['read', 'search']
---
Check Controller → Service → Repo,
tests and secrets. Never edit files.
```

- A persona with its own instructions
- Limited tools: this one cannot write
- Own context: returns only a summary
- Answers as Blockers / Suggestions / Nits
- Claude Code: `.claude/agents/reviewer.md`

Prompts and commands

```
.github/prompts/  
pr-description.prompt.md  
---  
description: Draft the PR description  
---  
Summarize this branch against main.  
## Context  
## Changes  
## How to test
```

- A saved prompt you launch by hand: /pr-description
- Copilot: .github/prompts/*.prompt.md
- Claude Code: .claude/commands/ (or a skill)
- Windsurf: .windsurf/workflows/*.md
- Manual trigger; a skill can trigger itself

Tools and hooks

```
.mcp.json
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

```
.claude/settings.json (hook)
"PostToolUse": [{
  "matcher": "Edit|Write",
  "hooks": [{ "type": "command",
    "command": "npm run lint" }]
}]
```

- MCP connects the agent to other systems: issues, database, browser
- Hooks run commands on events: after an edit, before a commit
- Instructions ask, hooks enforce
- Never commit tokens: use environment variables

CI/CD: hooks are CI on your laptop

SECTION 04

Providers

Same concepts, different folders

Concept	GitHub Copilot	Claude Code	Windsurf / Devin
Main file	.github/copilot-instructions.md + AGENTS.md	CLAUDE.md	AGENTS.md + .windsurf/rules/
Scoped rules	.github/instructions/*.instructions.md	Nested CLAUDE.md, .claude/rules/	Rules with “glob” trigger
Skills	.github/skills/	.claude/skills/	.windsurf/skills/
Agents	.github/agents/*.agent.md	.claude/agents/	—
Prompts	.github/prompts/*.prompt.md	.claude/commands/	.windsurf/workflows/
MCP	.vscode/mcp.json	.mcp.json	User-level config

Main differences

GitHub Copilot

Lives where the code lives

- IDE, CLI and GitHub.com
- Coding agent: issue → PR
- Automatic PR code review
- Several models to choose

Claude Code

Terminal-first agent

- CLI, IDE and desktop
- Long multi-step tasks
- Subagents, hooks, plugins
- Claude models

Windsurf

AI-native editor (now Devin)

- VS Code-based editor
- Rule activation modes
- Character limits per rule file
- Local memories (not shared)

Cursor

AI-native editor

- VS Code-based editor
- Rules in .cursor/rules/*.mdc
- Reads AGENTS.md
- Several models to choose

One source of truth

```
AGENTS.md           ← real content
CLAUDE.md           ← one line:
  @AGENTS.md
  (or: ln -s AGENTS.md CLAUDE.md)
.github/
  copilot-instructions.md ← optional
  skills/                 ← shared skills
```

- Each teammate may use a different tool
- Write the content once in AGENTS.md
- Thin adapters per tool: import or symlink
- Keep skills in one folder
- Three files saying different things = confusion

AI configuration is code

Pull request

Changes to AGENTS.md or skills go through a PR, like code.

Review

Copilot can review, a human must approve (1 approval rule).

CI/CD

The agent runs the same commands as the pipeline before the PR.

Scrum

Retro → update rules. DoD written in AGENTS.md.

It lives in the repo, it is reviewed, and it improves every sprint.

Good practices

You sign every line the AI writes.

- Review AI output like a teammate's PR
- No secrets in instruction files or MCP configs
- Issues, web pages or files can hide instructions (prompt injection)
- Stale rules produce wrong code: keep them current
- Follow the course AI usage policy

Your turn

```
git switch -c feature/ai-config

# 1. AGENTS.md
  overview, commands, conventions
  Definition of Done
# 2. One scoped rule
  frontend or backend
# 3. One skill
  a task you repeat
# 4. Optional: reviewer agent
  read-only tools
# 5. Open the PR
  a teammate reviews and approves

Test it: "How do I run the tests?"
```

- Work as a team, one PR per group
- Use the AI tool each of you prefers
- The answer must come from your file, not a guess
- Squash merge when approved

25 minutes · the PR is today's deliverable

Questions?

RESOURCES

[agents.md](#)

docs.github.com/copilot

code.claude.com/docs

docs.windsurf.com

prieraa@tecnocampus.cat · jlopeza@tecnocampus.cat

Software Lab II · Week 02 · 103411 · Tecnocampus UPF



Centre universitari adscrit a la

